

RETAILMART V3 • SQL

earlier sessions Review & Practice

WhatsApp Announcement

earlier sessions Review & Practice

No new concepts today. Instead, 8 mixed-difficulty problems covering EVERYTHING from earlier sessions. Multi-table JOINS, CTEs, subqueries, CASE WHEN, EXISTS, self-joins, UNION ALL, and debugging.

Problem 1: Multi-table JOIN with filtering & aggregation

Problem 2: CTE chain (filter -> group -> rank)

Problem 3: Subquery vs JOIN rewrite

Problem 4: CASE WHEN inside aggregate + GROUP BY

Problem 5: EXISTS with date filtering

Problem 6: Self-join with aggregation

Problem 7: UNION ALL with consistent column types

Problem 8: Debug a broken query (find & fix 3 errors)

This is your checkpoint before Window Functions start on Window Functions Part 1. Every problem here combines concepts from multiple days. If any problem feels hard, review that day's slides.

earlier sessions Recap

Everything you have learned so far:

- earlier: DDL, data types, constraints, INSERT/UPDATE/DELETE, filtering, ORDER BY
- earlier: Scalar functions, CASE WHEN, COALESCE, aggregates, GROUP BY, HAVING, JOINS (INNER, LEFT, RIGHT, FULL)
- earlier: Self-join, UNION, subqueries (scalar, IN, correlated), EXISTS, CTEs, recursive CTEs, normalization, ACID, transactions

Today we combine ALL of these into 8 mixed-difficulty problems. No new syntax. Pure practice.

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Block 1: JOINS + Aggregation	30 min	Problem 1: Multi-table JOIN with filtering, Problem 2: CTE chain (filter -> group -> rank)
Block 2: Subqueries + Logic	30 min	Problem 3: Subquery vs JOIN rewrite, Problem 4: CASE WHEN inside aggregate
Block 3: Advanced Patterns	30 min	Problem 5: EXISTS + date filtering, Problem 6: Self-join with aggregation
Block 4: Set Ops + Debugging	30 min	Problem 7: UNION ALL with matching types, Problem 8: Debug a broken query (3 errors)

YOUR SQL JOURNEY



SQL Intro



Basic Queries



Data Types



Constraints



Filtering



Scalar Functions



Conditional Logic



Aggregates



Joins Part 1



Joins Part 2



Self, Cross & Sets



Subqueries Part 1



CTEs & Correlated



DB Concepts



Review & Practice

← **HERE**

The Analyst's First Day on the Job

You have just been hired as a junior data analyst at RetailMart. Your manager drops 8 business questions on your desk and says: 'I need answers by end of day. Use the database.'

No one tells you which table to query or which JOIN to use. No one says 'use a CTE here.' You must read the question, identify the tables, choose the right approach, and write clean SQL.

This is what real analyst work looks like. Today's 8 problems simulate your first day. Each question requires combining multiple concepts from earlier sessions.

Revenue by Region for Delivered Orders in 2025

MEDIUM

SCENARIO: The CFO is preparing the 2025 annual report. She needs total revenue broken down by region -- but only for orders that were actually delivered. Cancelled and processing orders should NOT count toward revenue.

TASK: Write a query that shows region, total_revenue, and order_count for all delivered orders in 2025. Join sales.orders with stores.stores (for region) and filter for status = 'Delivered' and order_date in 2025. Sort by revenue descending.

HINT: Three filtering moments in one query — pre-aggregation (WHERE for status + date), grouping (GROUP BY region), and ordering. Trap: 'Region' isn't on stores directly — chain through core.dim_region; using a hardcoded city string instead of region_id silently miscounts.

Answer

 ANSWER

```
SELECT dr.region_name,  
       SUM(o.net_total) AS total_revenue,  
       COUNT(*) AS order_count  
FROM sales.orders o  
JOIN stores.stores s  
  ON o.store_id = s.store_id  
JOIN core.dim_region dr  
  ON s.region_id = dr.region_id  
WHERE o.order_status = 'Delivered'  
      AND o.order_date >= '2025-01-01'  
      AND o.order_date < '2026-01-01'  
GROUP BY dr.region_name  
ORDER BY total_revenue DESC;
```


Top 3 Stores by Average Order Value (2025, Delivered Only)

HARD

SCENARIO: The operations team wants to identify which stores have the highest average order value -- but only for delivered orders in 2025. They want the top 3 stores with their store name, city, and average order value. Build it as a pipeline: filter first, then aggregate, then rank.

TASK: Write a 3-step CTE pipeline. Step 1 (filtered_orders): filter orders to Delivered + 2025. Step 2 (store_aov): compute average order value per store_id. Step 3 (outer query): join with stores for name/city, sort by avg_order_value DESC, LIMIT 3.

HINT: Multi-step ranking lives naturally in a CTE chain — each step builds on the previous one. Trap: 'top 3 per group' needs ranking logic; without window functions (Window Functions Part 1), use a derived-table MAX trick OR a correlated subquery — both work, but each has different performance characteristics on large data.

Answer (1/2)

✓ ANSWER

```
WITH filtered_orders AS (  
    SELECT store_id, net_total  
    FROM sales.orders  
    WHERE order_status = 'Delivered'  
        AND order_date >= '2025-01-01'  
        AND order_date < '2026-01-01'  
),  
store_aov AS (  
    SELECT store_id,  
        ROUND(AVG(net_total), 2)  
        AS avg_order_value,  
        COUNT(*) AS order_count  
    FROM filtered_orders  
    GROUP BY store_id  
)  
SELECT s.store_name, s.city,  
    sa.avg_order_value,  
    sa.order_count  
FROM store_aov sa  
JOIN stores.stores s  
    ON sa.store_id = s.store_id  
ORDER BY sa.avg_order_value DESC
```

Answer (2/2)

 ANSWER

LIMIT 3;

Customers Who Spent Above Average -- Two Ways

MEDIUM

SCENARIO: The marketing team wants customers whose total lifetime spend exceeds the company-wide average spend per customer. You must solve this TWO ways: once with a subquery, once with a JOIN. Then explain which you would use in production.

TASK: First, write the subquery version: use a scalar subquery in HAVING to compare each customer's total to the average. Then rewrite it using a CTE and a CROSS JOIN. Return `customer_id`, `total_spend`, and the average for comparison.

HINT: Same business question, two SQL shapes. Subquery version: scalar (`SELECT AVG ...`) inside WHERE. JOIN version: pre-compute the average in a CTE/derived table, then CROSS JOIN to every customer row. Trap: the JOIN version produces the same numeric answer but is often FASTER on large data because the average is computed once, not re-evaluated per row.

Answer -- Subquery Version

✓ ANSWER

```
-- VERSION 1: Subquery in HAVING
SELECT cust_id,
       SUM(net_total) AS total_spend
FROM sales.orders
GROUP BY cust_id
HAVING SUM(net_total) > (
    SELECT AVG(cust_total)
    FROM (
        SELECT SUM(net_total) AS cust_total
        FROM sales.orders
        GROUP BY cust_id
    ) sub
)
ORDER BY total_spend DESC;
```

Answer -- CTE + CROSS JOIN Version



ANSWER

```
-- VERSION 2: CTE + CROSS JOIN
WITH customer_totals AS (
    SELECT cust_id,
           SUM(net_total) AS total_spend
    FROM sales.orders
    GROUP BY cust_id
),
avg_spend AS (
    SELECT AVG(total_spend) AS avg_total
    FROM customer_totals
)
SELECT ct.cust_id, ct.total_spend,
       ROUND(a.avg_total, 2) AS company_avg
FROM customer_totals ct
CROSS JOIN avg_spend a
WHERE ct.total_spend > a.avg_total
ORDER BY ct.total_spend DESC;

-- Production pick: CTE version
-- More readable, reusable, easier to debug
```

Order Status Breakdown per Store

MEDIUM

SCENARIO: The operations dashboard needs a single row per store showing how many orders are Delivered, Processing, and Cancelled. Not 3 rows per store with GROUP BY status -- one row with 3 count columns side by side.

TASK: Write a query that returns store_name, delivered_count, processing_count, cancelled_count, and total_count. Use CASE WHEN inside COUNT() to create conditional columns. JOIN with stores for the name.

HINT: 'Different metric per status, in one row' = conditional aggregation. Wrap a CASE inside COUNT/SUM/AVG so only matching rows contribute. Trap: COUNT(CASE WHEN x THEN 1 ELSE 0 END) counts ALL rows because 0 isn't NULL — drop the ELSE so non-matches become NULL and skip.

Answer



ANSWER

```
SELECT s.store_name,  
       COUNT(CASE WHEN o.order_status = 'Delivered'  
                   THEN 1 END) AS delivered_count,  
       COUNT(CASE WHEN o.order_status = 'Processing'  
                   THEN 1 END) AS processing_count,  
       COUNT(CASE WHEN o.order_status = 'Cancelled'  
                   THEN 1 END) AS cancelled_count,  
       COUNT(*) AS total_count  
FROM sales.orders o  
JOIN stores.stores s  
  ON o.store_id = s.store_id  
GROUP BY s.store_name  
ORDER BY total_count DESC;
```


Customers with No Orders in Last 6 Months

HARD

SCENARIO: The retention team defines 'churned customers' as those who placed at least one order historically but have NOT ordered in the last 6 months. They need names and emails for a win-back campaign.

TASK: Find customers who have at least one order in sales.orders BUT do NOT have any order with order_date >= CURRENT_DATE - INTERVAL '6 months'. Use EXISTS for the first check and NOT EXISTS for the second. Return first_name, last_name, email.

HINT: Two EXISTS-shaped conditions — one positive ('has SOME orders'), one negative ('NO recent orders'). Both are correlated. Trap: using NOT IN instead of NOT EXISTS for the recent-orders check breaks silently if any cust_id is NULL — NOT EXISTS handles NULLs correctly.

Answer

✓ ANSWER

```
SELECT c.first_name, c.last_name, c.email
FROM customers.customers c
WHERE EXISTS (
    SELECT 1 FROM sales.orders o
    WHERE o.cust_id = c.customer_id
)
AND NOT EXISTS (
    SELECT 1 FROM sales.orders o
    WHERE o.cust_id = c.customer_id
    AND o.order_date >= CURRENT_DATE
        - INTERVAL '6 months'
)
ORDER BY c.last_name, c.first_name;
```

```
-- EXISTS = has ordered at least once (ever)
-- NOT EXISTS = no order in recent 6 months
-- Combined = churned customer
```

Store Managers Compared to Their Store's Average Salary

HARD

SCENARIO: HR wants every employee with the role 'Store Manager' to be compared against their store's NON-MANAGER team. For each manager: show their store, their salary, the team size (everyone else at that store), and the team's average salary. This is the SELF JOIN pattern adapted to RetailMart's schema (which has no manager_id column — we use role + store_id instead).

TASK: SELF JOIN stores.employees: one alias for the manager (role ILIKE '%Manager%'), another for their store-mates (same store_id, NOT a manager). LEFT JOIN to keep managers with no team. GROUP BY manager. Aggregate team size and average team salary.

HINT: Two aliases on the same table — one filtered to managers, the other to non-managers at the same store. Trap: a strict INNER JOIN would silently drop solo managers (zero team) from the report; LEFT JOIN preserves them with team_size = 0. Also use COUNT(emp.employee_id) not COUNT(*) so zero-team managers show 0, not 1.

Answer



ANSWER

```
SELECT
  mgr.employee_id AS manager_id,
  mgr.first_name || ' ' || mgr.last_name
    AS manager_name,
  mgr.store_id,
  mgr.salary AS manager_salary,
  COUNT(emp.employee_id) AS team_size,
  COALESCE(ROUND(AVG(emp.salary)), 0)
    AS avg_team_salary,
  mgr.salary - COALESCE(ROUND(
    AVG(emp.salary), 0) AS pay_premium
FROM stores.employees mgr
LEFT JOIN stores.employees emp
  ON emp.store_id = mgr.store_id
  AND emp.role NOT ILIKE '%manager%'
  AND emp.employee_id != mgr.employee_id
WHERE mgr.role ILIKE '%manager%'
GROUP BY mgr.employee_id,
  mgr.first_name, mgr.last_name,
  mgr.store_id, mgr.salary
ORDER BY team_size DESC LIMIT 15;
```

Combined Activity Log from Two Audit Tables

MEDIUM

SCENARIO: The security team wants a combined activity feed from two different audit tables: `audit.application_logs` and `audit.api_requests`. Each table has different columns, but the security team wants one unified feed with: `timestamp`, `source_table`, `severity_or_method`, and `message_or_endpoint`.

TASK: Use `UNION ALL` to stack both tables. Map each table's columns to the same 4-column output: `event_time`, `source`, `type`, `detail`. Add a literal string column to identify which table each row came from. Order by `event_time DESC`, `LIMIT 100`.

HINT: Three iron rules — same column COUNT, compatible TYPES in matching positions, names from FIRST query. Trap: `api_requests` has a numeric `status_code` while `application_logs` has a 'level' string — direct `UNION` will fail with a type mismatch; cast both to `TEXT` or pick the same compatible representation in each `SELECT`.

Answer



```
SELECT al.timestamp AS event_time,  
       'app_log' AS source,  
       al.level AS type,  
       al.message AS detail  
FROM audit.application_logs al
```

```
UNION ALL
```

```
SELECT ar.timestamp AS event_time,  
       'api_request' AS source,  
       ar.method AS type,  
       ar.endpoint AS detail  
FROM audit.api_requests ar
```

```
ORDER BY event_time DESC  
LIMIT 100;
```

```
-- UNION ALL = keep all rows (faster)  
-- UNION = deduplicate (unnecessary here,  
--    rows come from different tables)
```

Fix the Broken Revenue-by-Brand Report

CRAZY

SCENARIO: A colleague wrote a query for 'total revenue per brand for 2025, showing only brands with revenue above 1 million.' The query runs but returns WRONG results. There are exactly 3 bugs. Find and fix all three.

TASK: Read the broken query below. Identify each of the 3 errors, explain WHY each is wrong, and provide the corrected query.

HINT: Three common bug patterns hide in this query — one calculation, one filter-placement, one boundary. Trap: revenue isn't unit_price alone — it's unit_price × quantity; mixing them up silently understates revenue without any error message.

Broken Query (3 bugs)

✓ ANSWER

```
-- BROKEN -- find and fix 3 errors
SELECT b.brand_name,
       COUNT(*) AS item_count,
       SUM(oi.unit_price) AS total_revenue
FROM sales.order_items oi
JOIN products.products p
  ON oi.prod_id = p.product_id
JOIN core.dim_brand b
  ON p.brand_id = b.brand_id
WHERE b.brand_name IN (
  SELECT b2.brand_name
  FROM core.dim_brand b2
  GROUP BY b2.brand_name
  HAVING SUM(oi.unit_price) > 1000000
)
AND oi.order_id IN (
  SELECT order_id FROM sales.orders
  WHERE order_date >= '2025-01-01'
)
GROUP BY b.brand_name
ORDER BY total_revenue DESC;
```


Answer -- 3 Bugs Identified & Fixed (1/2)

✓ ANSWER

```
-- BUG 1: Revenue calculation is WRONG
-- SUM(oi.unit_price) ignores quantity!
-- A unit_price of 500 × 10 items = 5000,
-- not 500. Fix: SUM(unit_price * quantity)

-- BUG 2: HAVING placed inside a subquery
-- that doesn't have oi in scope properly
-- The threshold belongs in the OUTER
-- GROUP BY's HAVING, not nested in WHERE-IN.
-- Fix: Drop the brand subquery; move the
-- > 1M filter to outer HAVING.

-- BUG 3: Date filter misses the end boundary
-- WHERE order_date >= '2025-01-01' captures
-- 2025 AND beyond. Fix: add an upper bound
-- AND order_date < '2026-01-01'.

-- CORRECTED QUERY:
SELECT b.brand_name,
       COUNT(*) AS item_count,
       SUM(oi.unit_price * oi.quantity)
       AS total_revenue
```

Answer -- 3 Bugs Identified & Fixed (2/2)

✓ ANSWER

```
FROM sales.order_items oi
JOIN products.products p
  ON oi.prod_id = p.product_id
JOIN core.dim_brand b
  ON p.brand_id = b.brand_id
JOIN sales.orders o
  ON oi.order_id = o.order_id
WHERE o.order_date >= '2025-01-01'
  AND o.order_date < '2026-01-01'
GROUP BY b.brand_name
HAVING SUM(oi.unit_price * oi.quantity)
  > 1000000
ORDER BY total_revenue DESC;
```

WHERE vs HAVING

Q: What is the difference between WHERE and HAVING?

A: WHERE filters individual rows BEFORE grouping. HAVING filters groups AFTER GROUP BY. WHERE cannot use aggregate functions. HAVING can. Example: WHERE salary > 50000 filters rows. HAVING COUNT(*) > 5 filters groups with more than 5 members.

INNER JOIN vs LEFT JOIN

Q: When would you use LEFT JOIN instead of INNER JOIN?

A: INNER JOIN returns only matching rows from both tables. LEFT JOIN returns ALL rows from the left table, with NULL for non-matching right-side columns. Use LEFT JOIN when you need to include entities with no match -- like customers with zero orders, or products never sold.

Subquery vs CTE

Q: When would you use a CTE instead of a subquery?

A: CTEs are named and readable -- they break a complex query into labeled steps. Subqueries are inline and compact. Use CTEs when: the logic has 2+ steps, you reference the same result multiple times, or you need recursion. Use subqueries for simple one-off checks (EXISTS, IN, scalar comparisons). Performance is usually identical.

EXISTS vs IN

Q: What is the difference between EXISTS and IN?

A: EXISTS checks if any row exists in the subquery (returns TRUE/FALSE). IN checks if a value matches any value in a list.

Key difference: EXISTS handles NULLs correctly. NOT IN with NULLs in the subquery returns no rows (a common bug).

NOT EXISTS always works. Performance: EXISTS can short-circuit (stop at first match). IN materializes the full list.

REAL WORLD

What You Practiced Today

8 Mixed-Difficulty Problems

Multi-table JOINS with aggregation, CTE pipelines, subquery vs JOIN rewrites, conditional aggregation (CASE inside COUNT), EXISTS for churn detection, self-joins for hierarchies, UNION ALL for unified feeds, and debugging broken queries.

Concepts Combined:

- JOINS (earlier sessions) + Aggregates (Aggregates & Grouping) + Filtering (Filtering & Sorting) in one query
- CTEs (CTEs) chaining filter -> aggregate -> rank steps
- CASE WHEN (Conditional Logic) inside aggregate functions for pivot-style output
- EXISTS/NOT EXISTS (CTEs) with date arithmetic (Scalar Functions)
- Self-JOIN (Subqueries Part 1) with GROUP BY for hierarchy analysis
- UNION ALL (Subqueries Part 1) for combining different-schema tables
- Debugging skills: wrong calculations, missing JOINS, filter placement

Next up (Window Functions Part 1): Window Functions Part 1 -- OVER(), PARTITION BY, ROW_NUMBER, RANK, DENSE_RANK, NTILE. A fundamentally new concept that gives you GROUP BY's power WITHOUT collapsing rows.

earlier sessions Review & Practice

-- Multi-Table JOIN + Aggregate

```
SELECT s.region, SUM(o.net_total)
FROM sales.orders o
JOIN stores.stores s
  ON o.store_id = s.store_id
WHERE o.order_status = 'Delivered'
GROUP BY s.region;
```

-- CTE Pipeline Pattern

```
WITH step1 AS (... filter ...),
     step2 AS (... aggregate ...)
SELECT ... FROM step2
JOIN ... ORDER BY ...;
-- Each CTE = one logical step
```

-- Conditional Aggregation

```
COUNT(CASE WHEN status='Delivered'
  THEN 1 END) AS delivered,
COUNT(CASE WHEN status='Cancelled'
  THEN 1 END) AS cancelled
-- One row per group, multiple counts
```


earlier sessions Review & Practice

-- EXISTS / NOT EXISTS

```
WHERE EXISTS (  
    SELECT 1 FROM orders o  
    WHERE o.cust_id = c.customer_id)  
AND NOT EXISTS (  
    SELECT 1 FROM orders o  
    WHERE o.cust_id = c.customer_id  
    AND o.order_date >= '2025-01-01')
```

-- Self-Join

```
FROM employees mgr  
LEFT JOIN employees emp  
    ON emp.manager_id = mgr.employee_id  
GROUP BY mgr.employee_id  
-- Same table, two roles (manager/report)
```

-- UNION ALL

```
SELECT col1, col2 FROM table_a  
UNION ALL  
SELECT col1, col2 FROM table_b  
-- Same column count + compatible types  
-- ALL = keep duplicates (faster)
```

CHALLENGE

CHALLENGE (1/3)

10 Mixed Practice Problems

CHALLENGE (2/3)

- 1.: Find the top 5 customers by total spend, showing first_name, email, total_spend, and order_count. (JOINS + aggregation)
- 2.: For each store, show the percentage of orders that are Delivered vs total orders. Use CASE WHEN inside COUNT. (Conditional aggregation)
- 3.: Find products that have NEVER been ordered. Use NOT EXISTS. Then rewrite the same query using LEFT JOIN WHERE ... IS NULL. (EXISTS vs JOIN)
- 4.: Build a CTE pipeline: get all 2025 orders, compute monthly revenue, then find the month with the highest revenue. (CTE chain)
- 5.: Find customers whose first order was in 2025 (they are new customers). Use a subquery with MIN(order_date). (Subquery + date)
- 6.: Show employees alongside their manager's name and salary. Flag employees who earn MORE than their manager. (Self-join + CASE WHEN)
- 7.:

CHALLENGE (3/3)

Combine support.tickets and sales.returns into one 'customer issues' feed with columns: issue_date, issue_type, customer_id, description. (UNION ALL)

8.: Find stores where the average order value is more than 20% above the company-wide average. (Subquery or CTE + HAVING)

9.: Find the employee with the highest salary in each department using a correlated subquery. Then rewrite using a CTE. (Correlated subquery vs CTE)

10.: (Crazy) Debug this query: `SELECT category, AVG(price) FROM products.products WHERE AVG(price) > 500 GROUP BY category` -- find the error, explain why it fails, and fix it.

YOUR SQL JOURNEY



SQL Intro



Basic Queries



Data Types



Constraints



Filtering



Scalar Functions



Conditional Logic



Aggregates



Joins Part 1



Joins Part 2



Self, Cross & Sets



Subqueries Part 1



CTEs & Correlated



DB Concepts



Review & Practice

← **HERE**